

Dispatch Grid

Complete Technical Report

A multi-agent AI swarm for disaster-time emergency dispatch: architecture, algorithms, and a file-by-file walkthrough of the full stack.

Ananya Soni | July 2026

github.com/AnanyaSoni2004/The-Smart-City-Dynamic-Dispatch-Grid

Contents

1. What this project is . . . 3
2. The full-stack architecture . . . 3
3. Repository map . . . 4

Part I — The simulation engine 4

4. models.py — the vocabulary . . . 5
5. callgen.py — the synthetic 911 stream . . . 6
6. triage.py — Agent 1: extraction, dedup, merge . . . 7
7. routing.py — the city graph and pathfinding . . . 8
8. dispatch.py — Agent 2: allocation, preemption, lifecycle . . . 8
9. coordinator.py — the swarm layer . . . 9
10. simulation.py + main.py — the batch driver . . . 10
11. llm_triage.py — the LLM-backed extractor . . . 10
12. evaluation.py — the honesty harness . . . 10

Part II — The web backend 12

13. server/engine.py — the tick-stepping session . . . 12
14. server/realmode.py — real cities and real calls . . . 13
15. server/db.py — persistence . . . 14
16. server/app.py — the FastAPI application . . . 14

Part III — The city data pipeline 16

17. scripts/build_city.py — OpenStreetMap -> committed JSON . . . 16

Part IV — The frontend 17

18. Foundations . . . 17
19. pages/Home.tsx . . . 17
20. pages/RunPage.tsx — the live/replay state machine . . . 18
21. components/CityMap.tsx — the hand-rolled map . . . 18
22. The panel components . . . 18

Part V — Deployment 20

23. Dockerfile — two stages . . . 20
24. Operational safeguards . . . 20

Design decisions at a glance . . . 20
How to extend the project . . . 21

This document explains the entire project: what it is, how every layer works, and what every file — and every significant line — does and why. It is written to be read top-to-bottom, but each part stands alone. Companion reading: **README.md** (architecture overview and schemas) and **CODE_WALKTHROUGH.md** (an earlier engine-only walkthrough).

1. What this project is

Dispatch Grid is a **multi-agent emergency-dispatch simulator** with a full web front end. The core problem it models: during a city-wide disaster, a 911 center receives far more calls than incidents (panicked people report the same fire dozens of times, with typos, wrong counts, and hedged rumors), while emergency units are scarce and roads keep closing. The system must:

1. **Extract structure from noise** — turn each raw transcript into a typed report (where, what, how bad, how many people). 2. **Deduplicate** — collapse hundreds of reports into tens of incidents. 3. **Suppress false reports** without ever ignoring a real one. 4. **Prioritize** by lives at risk, with anti-starvation aging. 5. **Dispatch** scarce units atomically over a live road graph, re-routing around closures and preempting low-priority missions when something worse happens.

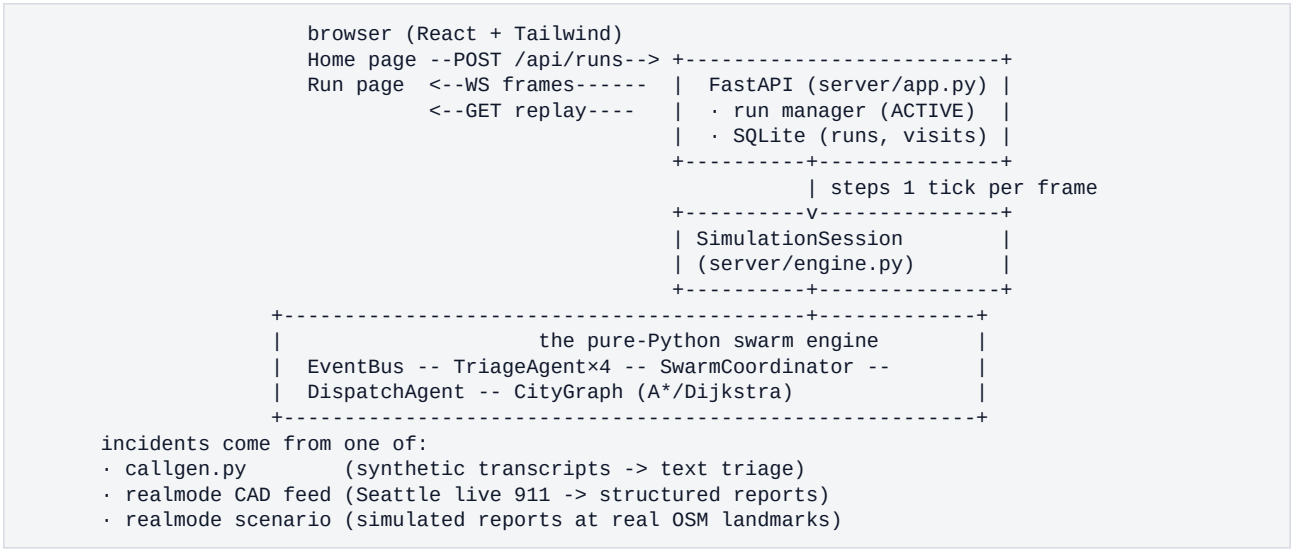
The engine is pure Python with **zero third-party dependencies**. On top of it sits a FastAPI backend that runs simulations server-side and streams every tick to a React dashboard over WebSocket; completed runs persist to SQLite behind permanent shareable URLs. Three scenario sources exist:

Scenario	Roads	Stations	Incidents
Synthetic grid city	generated 12×12 grid	8 fixed nodes	1,000+ generated noisy transcripts
Seattle US	real (OpenStreet Map)	36 real fire stations	**real live 911 calls** (data.seattle.gov)
Delhi / Mumbai IN	real (OpenStreet Map)	34 / 37 real fire stations	simulated at real landmarks (no public feed exists in India)

Two principles organize the whole codebase:

- **Single-writer state.** Every module owns exactly one kind of state (triage agents own nothing, the coordinator owns incidents, the dispatch agent owns units, the graph owns roads) and modules communicate only through typed messages. No locks, no races — conflicts are impossible *by construction*.
- **Ground truth is quarantined.** The synthetic call generator knows which calls describe the same real event and which are fake, but those fields are read only by the scoring code. The agents work from noisy text alone, like production would.

2. The full-stack architecture



Lifecycle of a run: `POST /api/runs` builds a `SimulationSession` and starts an asyncio task. The task calls `session.step()` once per tick — each step ingests due calls, advances every agent, and returns a JSON *frame* (metrics, incidents, unit positions, closures, events). Frames are pushed to every connected WebSocket, paced so the browser sees a smooth live animation. When the simulation drains, the full frame list is written to SQLite and the run URL becomes a permanent scrub-able replay.

3. Repository map

```
dispatch_grid/
+-- dispatch_grid/          the pure-Python swarm engine (Part I)
|   +-- models.py           every dataclass/enum — the system vocabulary
|   +-- callgen.py          synthetic 911 stream generator + gazetteer
|   +-- triage.py           Agent 1: extraction, dedup scoring, merge
|   +-- routing.py          city graph, Dijkstra, A*, dynamic re-routing
|   +-- dispatch.py         Agent 2: allocation, preemption, unit lifecycle
|   +-- coordinator.py      event bus, quarantine, priority queue, metrics
|   +-- simulation.py       batch event loop + console dashboard
|   +-- main.py             CLI entry point
|   +-- llm_triage.py       LLM-backed triage (same contract, Claude API)
|   +-- evaluation.py       extractor scoring vs labeled ground truth
+-- server/                 FastAPI web backend (Part II)
|   +-- engine.py           tick-stepping wrapper + frame builder
|   +-- realmode.py         real cities: OSM graphs, live CAD feed, scenarios
|   +-- app.py              REST + WebSocket + static serving + analytics
|   +-- db.py               SQLite persistence (runs, visits)
|   +-- data/*.json         committed city graphs (seattle/delhi/mumbai)
+-- scripts/build_city.py   OSM -> city JSON pipeline (Part III)
+-- web/                    React + Vite + Tailwind frontend (Part IV)
|   +-- src/{pages,components,api.ts,types.ts,theme.ts}
+-- Dockerfile              two-stage build (Node -> Python) for deployment
+-- render.yaml             Render.com blueprint
+-- dashboard.html          legacy self-contained in-browser demo (JS port)
+-- triage_eval.html        legacy in-browser LLM-vs-rules eval app
```

Part I — The simulation engine

Everything in `dispatch_grid/` is dependency-free Python. It can run headless (`python -m dispatch_grid.main`) with no web stack at all.

4. `models.py` — the vocabulary

This file imports nothing but the standard library, and everything else imports it — the dependency graph is a clean tree.

The enums. `IncidentType` (Fire, Medical, Accident, Flood, Building Collapse, Hazardous Material, Unknown) classifies what happened. `IncidentStatus` is the incident lifecycle: `pending -> dispatched -> on_scene -> resolved` (or `false_report`). `UnitType` is the five fleets (Ambulance, FireTruck, PoliceUnit, HazmatTeam, RescueBoat). `UnitStatus` is the unit lifecycle: `available -> en_route -> on_scene -> returning -> refueling -> available`. All enums subclass `str` so they JSON-serialize as their value with no custom encoder.

`RESOURCE_PROFILE` maps each incident type to its base unit needs — e.g. a Building Collapse needs `{FireTruck: 2, Ambulance: 2, Police: 1}`. Severity scales these later (see triage).

The counters.

```
_call_seq = itertools.count(1)
_inc_seq  = itertools.count(1)
```

Process-wide monotonic counters give calls IDs like `CALL00042` and incidents IDs like `INC0007`. They are module-level, so in the long-lived web server IDs keep growing across runs — harmless, and it guarantees no collision inside a run.

The dataclasses, each corresponding to one message in the architecture:

- `EmergencyCall` — raw input: `call_id`, `transcript`, `received_at` (sim seconds), plus two *hidden ground-truth* fields (`truth_incident_key`, `truth_is_false_report`) read only by evaluation code.
- `TriageReport` — Agent 1's structured output: `location` (canonical landmark string or None), `node` (resolved graph node or None — unroutable until known), `incident_type`, `severity` 1–5, `affected_people`, `resources_needed` (dict of UnitType->count), `urgency` 0.5–2.0 (language-cue multiplier), `confidence` 0–1 (drives false-report gating).
- `Incident` — the merged record. Adds bookkeeping the report lacks: `report_count` (corroboration), `first_reported`/`last_reported`, `assigned_units`, and the timestamp trio `dispatch_time`/`arrival_time`/ `resolve_time` used for response-time metrics.
- `Unit` — a resource row: current `node`, `home_node`, `status`, `fuel` (0–1), `assigned_incident`, `eta`, and its current `route` (list of node IDs). The property `dispatchable` requires `AVAILABLE` status *and* fuel > 0.15, so near-empty units rest until refueled.
- `DispatchOrder` — the final product: unit IDs, per-unit routes, worst-case ETA in minutes, and `preempted_from` if any unit was stolen from a lesser incident.

The most consequential lines in the file — `Incident.priority()`:

```
def priority(self, now: float) -> float:
    lives = max(1, self.affected_people)
    lives_factor  = 1.0 + (lives ** 0.5)                    # diminishing returns
    wait          = max(0.0, now - self.first_reported)
    aging         = 1.0 + min(1.0, wait / 600.0)            # +100% after 10 min
    corroboration = min(1.5, 0.8 + 0.1 * self.report_count)
    return self.severity * lives_factor * self.urgency * aging * corroboration
```

Line by line: `lives_factor` uses a square root so 100 people at risk don't drown out everything else (10× the people ~ 3.2× the factor, not 10×). `aging` grows linearly to a hard cap of 2.0 after ten minutes waiting — this is the anti-starvation term that guarantees a severity-2 incident eventually outranks fresh severity-3s. `corroboration` starts below 1.0 for a single-report incident (0.9) and rewards multiply-reported incidents up to 1.5× — an incident five people phoned in is more certainly real. The product form means any one factor can dominate: a mass-casualty collapse beats everything young, but nothing starves forever.

5. `callgen.py` — the synthetic 911 stream

The gazetteer. `LANDMARKS` maps 24 named places ("Central Mall", "Riverside Bridge"...) to a grid node and a list of misspelled/colloquial aliases ("central shoping mall", "the big mall"). This is the *only* location vocabulary the router can act on, which is exactly how real CAD systems work (dispatchers geocode against a gazetteer, not free text).

`TYPE_PHRASES`, `PANIC`, `TRAPPED`, `TYPO` are the phrase banks that make transcripts feel real: type-specific descriptions ("flames everywhere"), panic prefixes ("Oh my god,"), urgency injections ("kids are inside!"), and a typo dictionary (`fire->fyre`, `people->ppl`).

`_noisify(text, rng)` walks the words of a finished transcript and, with 25% probability, swaps a word for its typo'd form; with 4% probability it *drops the word entirely* (line `if rng.random() < 0.04: continue`). Dropped words simulate garbled audio and stress the extractor's tolerance for incomplete sentences.

`GroundTruthIncident` is the hidden answer key: one real event with a key (`GT0042`), type, landmark, severity, people count, and start time.

`CallGenerator.__init__` creates `n_incidents` ground-truth events:

```
sev    = rng.choices([1,2,3,4,5], weights=[10,22,30,25,13])[0]
people = max(0, int(rng.gauss(sev * 6, sev * 3)))
t      = rng.betavariate(2, 2) * duration
```

Severity follows a realistic bell (3 is most common, 5 rare). People scale with severity but with high variance. The Beta(2,2) start-time distribution peaks mid-simulation — the disaster *escalates* and then tapers, rather than arriving uniformly.

`_transcript(gt, hide_loc, second)` assembles one call: a panic prefix, the type phrase, and the location — which 50% of the time uses an *alias* rather than the canonical name (that's what makes fuzzy matching necessary). With 60% probability an urgency phrase is appended; if the incident is big, a **wrong people-count** is injected (`gt.people * uniform(0.4, 1.8)`) so different calls about the same event conflict, which the merge logic must reconcile. `second` occasionally appends a *second incident sighting* to one call — a caller reporting two things at once, which the (single-report) triage contract will under-extract; the duplicate stream covers for it.

`generate()` is the outer loop. For each ground-truth event it draws a duplicate count from `[1,2,3,4,6,9]` weighted toward small numbers (most events get 1–3 calls; a few get 9 — the "everyone phones about the mall fire" effect). Each duplicate is timestamped `gt.start + |gauss(0, 90)| + d * uniform(5, 40)`: calls cluster just after the event and trickle afterwards. 12% of calls hide their location (`hide = rng.random() < .12`) — those incidents stay unroutable until a later duplicate supplies it. Finally, 7% false reports are appended, written with deliberate hedging ("I think theres a ... ? not sure, my friend told me") so a good confidence model scores them low. The list is sorted by time and returned.

6. `triage.py` — Agent 1: extraction, dedup, merge

`TriageAgent` is stateless with respect to calls (any agent can process any call) — the swarm scales horizontally by construction.

Constructor. Pre-computes `_alias_table`: a flat list of `(normalized_alias, canonical_name, node)` for every landmark and alias — one linear structure to fuzzy-match against.

`extract(call) -> TriageReport`, step by step

1. **Normalize**: `_norm` lowercases and strips non-alphanumerics, so "FYRE!!" matches "fyre". 2. **Type vote**: counts keyword hits per incident type (`scores = {t: sum(1 for k in kws if k in text)}`); "rubble"/"caved in" add +2 to Collapse because collapse transcripts often also say "injured", which would otherwise tip the vote to Medical. No hits at all -> `UNKNOWN`. 3. **Location** (`_match_location`): first tries exact substring containment of any alias (score 1.0). Failing that, uses `SequenceMatcher.find_longest_match` — the longest common block between alias and transcript, scored as `block_size / max(8, len(alias))`, accepted above 0.72. The `max(8, ...)` denominator stops 3-letter aliases from matching everything. Returns `(canonical, node, score)` — the *canonical* name, so "the big mall" and "central shoping mall" dedup to the same location string. 4. **People**: an explicit number in the text wins (`NUM_RE`); otherwise a per-type prior (Fire->8, Collapse->15, Medical->1...), capped at 500. 5. **Urgency**: `1.0 + 0.25 × (urgency-cue hits)`, capped at 2.0 — "trapped", "screaming", "kids" each push it up. 6. **Severity**: a per-type base (Collapse 5, Fire 4, Medical 3...) adjusted ±1 by urgency cues, big people counts, and *down* by hedging ("not sure", "my friend told me") — a hedged fire is probably smaller than claimed. 7. **Confidence** — the false-report weapon:

``python confidence = max(0.05, min(1.0, 0.35 + 0.25 * loc_score + 0.15 * min(2, type_hits) - 0.2 * hedges)) ``

Base 0.35; a grounded location adds up to 0.25; clear type keywords up to 0.30; each hedge subtracts 0.20. A hedged, secondhand, vague call lands well below the coordinator's 0.42 gate; a firsthand report with a real landmark lands far above it. 8. **Resources** (`_scale_resources`): starts from `RESOURCE_PROFILE[type]` and multiplies counts when severity >= 4 or people > 25 — a sev-5 collapse asks for 4 fire trucks + 4 ambulances, not 2+2.

Dedup: `similarity(report, incident, now)`

```
s = 0.45·(same node) + 0.30·(same type) + 0.25·max(0, 1 - deltat/window)
```

Location is the strongest signal (0.45), type next (0.30), and recency decays linearly over a 15-minute window (0.25). Two hedged epsilon-cases: if *either* side has no location, a weak 0.15 benefit-of-the-doubt applies (else no-location reports could never merge), and Unknown type gets 0.10 partial credit against any type. The merge threshold is **0.62** — location+type alone (0.75) clears it; type+recent alone (<=0.55) does not. Two simultaneous fires at the same landmark are *intentionally* indistinguishable — merging them is the correct call until field units report otherwise.

`merge_or_create(report, active, now)`

Scans all active incidents for the best similarity; above threshold it merges, otherwise creates a new `Incident`. `_merge` **line by line:** `report_count` increments (feeding the corroboration priority term); severity and urgency take the **max** (never let a milder duplicate downgrade a bad incident); people-count blends `0.6·old + 0.4·new` (corroborated value drifts toward the consensus rather than jumping); a missing location

fills in from the new report (this is what un-blocks unroutable incidents); resource needs take the per-type max; and confidence rises with each corroboration (`+0.5·rep.confidence·0.4`, capped at 1.0) — the mechanism by which quarantined reports get released.

7. `routing.py` — the city graph and pathfinding

`CityGraph.__init__` builds a 12×12 grid: node `n = y*width + x`, edges to the right and down neighbors with weight `45s × uniform(0.8, 1.6)` (blocks are not identical), plus ~12 random diagonal "arterial" shortcuts at 0.7× the euclidean cost — they make routing non-trivial (the best path is often not the grid path).

Live conditions. `closed` is a set of undirected edge keys (`_key` orders the pair so (a,b) == (b,a)); `congestion` maps edge -> multiplier >= 1. `edge_time(a, b)` is the single choke point every router uses: returns `None` if closed, else `base × congestion`. Because both A* and Dijkstra call it per-edge, a closure injected mid-run instantly affects all future routing with no cache invalidation anywhere.

`dijkstra(src, dst)` is textbook with a lazy-deletion heap (`seen` set instead of decrease-key), early exit on reaching `dst`, and path reconstruction by walking `prev` backwards.

`astar(src, dst, min_block=30)` adds the heuristic `h(n) = euclid(n, dst) × 30`. Admissibility: no edge can be traversed faster than 30 s per unit of coordinate distance (grid blocks cost >= 45×0.8 = 36 s; real-city coords are km with a fastest road of 90 km/h = 40 s/km), so A* stays optimal. `route()` tries A* first and falls back to Dijkstra — belt and braces for pathological heuristic cases.

`reroute_if_blocked(current, dst, old_route)` is called every tick for every moving unit. It walks the *remaining* portion of the old route summing `edge_time`; if any edge is now closed (`ok=False`) or a freshly computed route is >40% faster (`t_new < t_old / 1.4`), the unit switches; otherwise it keeps its plan. The 1.4 hysteresis prevents units from thrashing between near-equal routes as congestion fluctuates.

8. `dispatch.py` — Agent 2: allocation, preemption, lifecycle

Constructor distributes the fleet round-robin across station nodes (`home = station_nodes[i % len(station_nodes)]`) and names units `FireTruck_3`-style.

`availability()` buckets every unit into available / dispatched / returning / out — this feeds the dashboard's fleet panel verbatim.

`try_dispatch(incident, now, allow_preempt, incidents)` — the heart

Read it as five rules enforced in order:

1. **No location, no dispatch**: `if inc.node is None: return None` — the coordinator holds it; a merged duplicate may fill the location in later. 2. **Per-type sourcing with coverage reserve**: for each required unit type, the dispatchable pool is computed and `ceil(fleet × 12%)` is held back (`usable = len(pool) - reserve`) — some of every fleet always stays home so a brand-new severity-5 doesn't find an empty garage. 3. **Preemption as a last resort**: only if the reserve-limited pool can't cover the need does `_preemptable()` add en-route units whose current incident's priority is less than `new_priority / 2.2`. The ratio (2.2×) is deliberately high: stealing a unit strands its old incident, so it must be *clearly* justified. 4. **Cheapest-first ranking**: candidates sort by `_cost` = routed travel time + `(1 - fuel) × 120` — a mild fuel penalty biases toward fresh units at equal distance. 5. **Atomicity**: if any required type can't be fully sourced — `if len(picks) < need: return None` — *nothing* is sent. Partial fills would waste scarce units standing idle at a

scene they can't handle; the incident stays queued and aging raises its priority instead. Unreachable units (`path is None` through closures) abort the same way.

The commit block then mutates world state in one place: preempted units are removed from their old incident's `assigned_units` (and the old incident flips back to `PENDING` if it lost everyone — it re-enters the queue automatically); each chosen unit becomes `EN_ROUTE` with its route and `eta = now + t`; the incident becomes `DISPATCHED`; and a `DispatchOrder` is returned carrying `preempted_from` for the metrics.

`tick(now, incidents, on_arrival, on_resolution)` — the world clock

A per-unit state machine, evaluated every tick:

- **EN_ROUTE**: first re-checks the route against new closures (`reroute_if_blocked`), updating route+ETA if switched. On `now >= eta` the unit arrives: status `ON_SCENE`, fuel -0.12, and — only for the *first* arriving unit (`inc.status == DISPATCHED` guard) — the incident flips to `ON_SCENE`, `arrival_time` is stamped, and `on_arrival` fires (response-time + lives-saved metrics). `return_eta = now + SCENE_TIME[severity]` — scene work scales 300 s (sev 1) to 1500 s (sev 5).
- **ON_SCENE** past `return_eta`: the incident resolves (once), and the unit routes home as `RETURNING`.
- **RETURNING** past `eta`: teleport-snap to `home_node`; below 30% fuel -> `REFUELING` for 600 s, else straight to `AVAILABLE`.
- **REFUELING** past `eta`: full tank, `AVAILABLE`.

9. `coordinator.py` — the swarm layer

`EventBus` is ~15 lines: `subscribe(topic, fn)` appends a callback, `publish(topic, payload, t)` logs the event and invokes subscribers synchronously. The log makes the whole run *replayable* — the web layer's event feed is derived from it. Topic names map 1:1 to Kafka/NATS topics in the production design (see README §4).

`SwarmCoordinator.__init__` wires itself to `calls.incoming` and `triage.report`, holds the single incident store (`self.incidents`), the quarantine list, and the metrics dict. `itertools.cycle` implements round-robin sharding across the triage swarm.

The triage path. `_on_call` picks the next agent (`self.triage_agents[next(self._rr)]`), extracts, and re-publishes as a `triage.report` — so structured reports from *any* source (rule-based, LLM, or the real-CAD path in the web layer) enter identically. `_on_report` applies the **false-report gate**:

```
if rep.confidence < 0.42 and not self._corroborates_existing(rep, t):
    self.quarantine.append((t, rep)); return
```

A low-confidence report that matches *nothing* active waits in quarantine. If a second independent report corroborates it within 420 s, `step()` releases and admits it; otherwise it silently expires. This bounds false-report damage to zero dispatched units, while guaranteeing any real incident reported twice gets through — the deliberate trade-off is a ~7-minute delay on real-but-hedged single reports.

`_admit` delegates dedup to `merge_or_create` against the global store and publishes `incident.created` or `incident.updated` accordingly. Note dedup always uses `triage_agents[0]` — the similarity function is stateless, so any agent works; using one keeps thresholds consistent.

`pending_queue(now)` heapifies `(-priority, id, incident)` triples (negated because Python's heap is a min-heap; the id breaks ties so incidents never compare) and pops into a sorted list — highest priority first.

`step(now)` runs the three phases in a fixed order every tick: (1) release or expire quarantined reports; (2) walk the priority queue calling `try_dispatch` on each pending incident (failures just stay queued — aging handles the rest); (3) `dispatch.tick(...)` advances the physical world, with `_on_arrival` recording response time and the lives-saved estimate:

```
frac = max(0.1, 1.0 - rt / 1800.0)
lives += people * frac * (severity / 5)
```

— the fraction of affected people saved decays linearly with response time, floored at 10%, weighted by severity. It's a *model*, clearly labeled as an estimate everywhere it's shown.

`snapshot(now)` packages counters, backlog, top-priority list, fleet availability, and average response minutes — the exact dict the web dashboard renders.

10. `simulation.py` + `main.py` — the batch driver

`build_system(seed)` wires graph + fleet (20 ambulances, 12 fire trucks, 35 police, 4 hazmat, 5 boats across 8 stations) + 4 triage agents + coordinator. `run(...)` generates calls, schedules 14 random road closures and 30 congestion waves over the run, then loops `t` from 0 to `duration + 1800` (the +1800 s drain lets in-flight incidents finish): each tick it publishes due calls, applies due disruptions, calls `coord.step(t)`, and every 300 s prints/records a snapshot. `main.py` is a thin argparse wrapper that writes the snapshot timeline to `timeline.json` (consumed by the legacy `dashboard.html`).

11. `llm_triage.py` — the LLM-backed extractor

`LLMTriageAgent(TriageAgent)` overrides *only* extraction — it inherits all dedup/merge machinery, proving the swarm is agnostic to which brain does the reading. Key mechanics, line by line:

- **The system prompt** embeds the gazetteer as a JSON list and demands strict JSON output: location must be one of those names *or null* ("never invent one") — hallucination control at the prompt level.
- `extract_batch` packs calls in chunks of 10 into single API requests (~10× cheaper/faster than per-call), zips responses back to calls by order, and falls back per-call to the rule-based extractor whenever a row is missing or malformed (`self._fallback.extract(c)`).
- `_call_llm` posts to the Anthropic Messages API with stdlib `urllib` (still zero dependencies), retries transient failures with linear backoff (`1.5 × (attempt+1)` seconds), strips markdown fences before parsing, and returns `None` on any failure — triggering fallback rather than crashing.
- `_to_report` is defense-in-depth validation: a location not in the gazetteer is nulled (hallucination control at the code level), unknown types map to `UNKNOWN`, and every numeric field is clamped to its legal range (`severity` 1–5, `people` 0–500, `urgency` 0.5–2.0, `confidence` 0–1).

The net property: an API outage degrades extraction quality; it can never halt dispatch.

12. `evaluation.py` — the honesty harness

Scores any extractor honoring `extract(call) -> TriageReport` against the generator's answer key, at two levels.

Field level (`score_extraction`): type accuracy; location accuracy (a correct landmark scores, and *correctly answering null* when the caller gave no location also scores — refusing to guess is an accuracy behavior); severity MAE and within-±1 rate; median relative people error; and **false-report discrimination** measured properly — `_auc` computes the Mann-Whitney probability that a random real call out-scores a random false report on confidence, and catch/loss rates at the coordinator's actual 0.42 gate tie the statistic to the system's real decision boundary.

System level (`score_system`): runs the *entire* swarm once per extractor on the same seed and compares end outcomes (dedup compression, backlog, response time, lives saved). The point: extraction errors only matter insofar as they change dispatch outcomes, and this measures exactly that.

Baseline (seed 42, 201 calls): 73.4% type accuracy, 92.6% location accuracy, severity MAE 1.21, false-report AUC 0.984 with 100% of false reports caught at the gate, at the cost of 6.4% of real calls (briefly) quarantined.

Part II — The web backend

The `server/` package turns the batch engine into a live, multi-user web service. Dependencies: `fastapi`, `uvicorn`, `pydantic` (see `requirements.txt`) — SQLite and everything else is stdlib.

13. `server/engine.py` — the tick-stepping session

`simulation.run()` executes an entire run at once; the web server needs to advance **one tick at a time** and emit a JSON *frame* per tick so it can stream a run live. `SimulationSession` mirrors `run()`'s setup exactly, then exposes `step()`.

Construction — three incident sources, one engine

```
if mode != "synthetic":
    city = realmode.load_city(mode)                 # committed OSM JSON
    self.graph = realmode.RealCityGraph(city)
    dispatch = realmode.RegionalDispatchAgent(self.graph, REAL_CITY_FLEET,
                                              [s["node"] for s in city["stations"]])
    self.coord = SwarmCoordinator([TriageAgent(...)×4], dispatch, bus)
    if mode in realmode.LIVE_FEED_CITIES:           # seattle
        max_calls = int(min(250, max(60, duration / 3600 * 200)))
        self.reports = realmode.build_reports(city, duration, max_calls)
    else:                                           # delhi, mumbai, ...
        self.reports = realmode.build_scenario_reports(city, duration,
                                                       n_incidents, seed)
else:
    self.graph, self.bus, self.coord = build_system()   # the original grid
    self.calls = CallGenerator(...).generate()
    # ...plus the 14 closures / 30 congestion waves, exactly as run() schedules them
```

The `max_calls` formula matters: the live feed compresses ~24 h of real calls into the chosen window, so demand must be scaled (~200 calls per simulated hour) or the fleet drowns — an earlier version without it ended runs with a backlog of 111.

`step()` — one tick

Ingest due synthetic *calls* onto `calls.incoming` (text triage path) **and/or** due pre-structured *reports* onto `triage.report` directly:

```
while self._ri < len(self.reports) and self.reports[self._ri].received_at <= t:
    self.coord.metrics["calls"] += 1          # _on_call normally counts this
    self.bus.publish("triage.report", {"report": ...}, t)
```

Real CAD records skip text triage (real transcripts are never public) but hit the identical bus topic, so dedup/quarantine/dispatch cannot tell the difference. Then due disruptions apply (synthetic mode only), `coord.step(t)` runs, `_frame(t)` is built, and time advances until `duration + 1800` (drain).

Motion interpolation — why units glide

The engine only updates `Unit.node` on *arrival*; between nodes a unit is "somewhere on its route". `_unit_position` reconstructs that for the map:

- `_motion[unit_id] = (depart_time, eta, from_node)` is armed the tick a unit transitions into `EN_ROUTE`/`RETURNING` (detected via `_prev_status`), and the stored `eta` is refreshed if mid-route re-routing

changed it.

- Progress is `frac = (now - depart) / (eta - depart)`, clamped to [0, 1].
- **En-route** units interpolate along their actual route polyline: `_along_route` scales `frac` across the route's node list, picks the segment (`i = int(frac × (len-1))`), and lerps within it.
- **Returning** units have no stored route (the engine doesn't keep it), so they lerp in a straight line home — a visual approximation that reads fine.

`_frame(now)` — the wire format

One frame = `t`, the metrics dict (from `coord.snapshot`, trimmed), fleet `resources`, active `incidents` (id/type/loc/node/sev/people/status/priority/ report-count/units — **resolved and false-report incidents are excluded**; they're not rendered and keeping them ballooned late-run frames; the resolved *count* travels in metrics), all 90+ `units` with interpolated `x, y` rounded to 3 decimals, `closed` edges, `congestion` triples, and `events` — new bus-log entries since the last frame, formatted human-readable ("INC0007 <- 3 unit(s), ETA 4.0 min"), capped at 12 per frame. `static_payload()` sends the immutable part once per run: node coordinates, edges, stations, and landmarks (the POI gazetteer for real cities; the synthetic gazetteer for the grid).

14. `server/realmode.py` — real cities and real calls

`RealCityGraph(CityGraph)`

Loads a committed city JSON *instead of* generating a grid (deliberately no `super().__init__`): `coords` in kilometers (top-left origin, y southward to match SVG), `adj` from `[a, b, seconds]` edge triples. `width/height` are the coordinate extents — the frontend scales its viewBox from them. The A* heuristic stays admissible because coords are km and the fastest road (90 km/h) costs 40 s/km > the 30 s/km heuristic. `reroute_if_blocked` **is overridden**: when there are no closures and no congestion (always true in real mode), the remaining route is still valid by definition, so it returns the remaining polyline + summed time and skips the expensive per-tick `route()` call — on an 8,000-node graph with 20 moving units, that skip is the difference between 9 ms and hundreds of ms per tick.

`RegionalDispatchAgent(DispatchAgent)`

Overrides only `_cost`: ranking candidate units by *exact* routed time means an A* run per candidate per incident — too slow at 8k nodes. The estimate `straight_line_km × 60 s/km + fuel_penalty` ranks the pool; the **chosen** unit's route is still computed exactly at dispatch time, so ETAs and movement stay truthful. Allocation is marginally less optimal; latency is ~50× better.

The Seattle live feed

`fetch_recent_calls()` GETs the Socrata endpoint (`data.seattle.gov/resource/kzjm-xkqj.json?$order=datetime DESC&$limit=400`) with a 3-minute in-process cache (several users starting runs together share one fetch). `_TYPE_RULES` maps CAD call types to engine types + severity by keyword, most-specific first — "multiple casualty" -> Medical/5 before the generic "medic" -> Medical/3; "fire in building" -> Fire/5 before "alarm" -> Fire/2. `_nearest_node` projects a call's lat/lon into the city's km frame using the stored projection constants and linear-scans for the closest node, rejecting calls > 2 km from any mapped road (the graph only carries major roads). `build_reports` filters to the city bbox, sorts by real timestamp, and maps timestamps onto `[0, 0.96 × duration]` **preserving relative**

spacing — the burstiness of the real night survives compression. Every record becomes a `TriageReport` with `confidence=0.95` (CAD-verified — never quarantined) whose duplicates (the feed re-lists updated incidents) exercise the dedup path with *genuinely real* duplicate data.

`build_scenario_reports` — cities without a feed

For Delhi/Mumbai (and any future city), incidents are generated at **real places**: 75% anchor to a random named POI from the city file (metro stations, hospitals, malls — so the map says "Kirti Nagar (Green Line)", not "node 4711"), the rest to random road nodes. Types follow a plausible urban distribution (40% medical, 22% accident, 20% fire...); severity is the type's base ±1. Crucially each incident emits **1–3 reports** with severity noise, people-count noise (`×uniform(0.6, 1.6)`), staggered timestamps (20–240 s apart) and confidence 0.6–0.95 — so the dedup/merge machinery works exactly as hard as it would on real duplicated calls. Deterministic per seed.

`available_cities()` simply globs `server/data/*_city.json` — dropping a new city file into the directory is the entire registration step. `LIVE_FEED_CITIES = {"seattle"}` is the one-line hook where a future live-feed adapter plugs in.

15. `server/db.py` — persistence

One SQLite file (`runs.db`), two tables, created idempotently on every connect (`CREATE TABLE IF NOT EXISTS` — no migration machinery needed at this scale):

- `runs`: id (8-hex-char primary key), created_at, params/summary/graph/ timeline as JSON text columns. `save_run` uses `INSERT OR REPLACE`; `get_run` rehydrates the JSON; `list_runs` returns metadata only (no timeline — the index endpoint must stay light).
- `visits`: ts, path, referrer, user-agent — the first-party analytics store. `visit_stats()` aggregates totals, daily counts (14 days), top paths, and top referrers in four small SQL queries.

16. `server/app.py` — the FastAPI application

Middleware: gzip (>2 KB — a multi-MB replay timeline compresses ~10×) and permissive CORS (useful during Vite dev; harmless in production since state- changing routes take JSON bodies).

`RunParams` is the validated input model: duration 300–7200 s, incidents 10–600, tick 5–60 s, `speed` 10–600 (sim-seconds per real second — the live pacing knob), and `mode` — `"synthetic"` or a real-city key, validated against `available_cities()` at request time so new city files work with no code change.

`ActiveRun` holds one in-flight run: the session, accumulated `frames`, `subscribers` (one asyncio.Queue per connected WebSocket), and `done/summary`. `ACTIVE` is a plain dict — uvicorn runs one process, and all mutation happens on the event loop, so no locking is needed.

`create_run` enforces `MAX_ACTIVE_RUNS = 4` (free-tier protection: strangers can't pile up CPU-burning simulations; the 429 message tells them to watch a replay), validates the mode, builds the session in a worker thread (`asyncio.to_thread` — construction fetches the live feed for Seattle), converts live-feed failures into a friendly 502, registers the run, and spawns `_run_loop` as a background task. Returns `{"id": ...}` immediately.

`_run_loop` is the heartbeat:

```
pace = tick / speed                       # real seconds per frame
while (frame := await asyncio.to_thread(run.session.step)) is not None:
    run.frames.append(frame)
    for q in run.subscribers: q.put_nowait({"type": "frame", "frame": frame})
    await asyncio.sleep(pace)
# then: summary, save to SQLite, broadcast {"type": "done"}, and
# finally: sleep 60 s before evicting from ACTIVE (late joiners still hit memory)
```

`step()` runs in a thread so a 9 ms engine tick never blocks other requests; `put_nowait` never awaits a slow client (their queue just grows). A default run (3600 s at 120×) streams for ~45 real seconds.

The WebSocket (`/api/runs/{id}/ws`) speaks a four-message protocol: `init` (params + static graph) -> `frames` (catch-up batches of 50, so a viewer joining mid-run fast-forwards instantly) -> `frame` (live, one per tick) -> `done` (summary). A connection to a non-active run gets `{"type": "error", "error": "not_active"}` — the client's cue to fetch the replay over REST instead.

Replay & index. `GET /api/runs/{id}` serves from memory while active, falling back to SQLite afterwards — the same URL transparently transitions from "live" to "permanent replay". `GET /api/runs` merges active + saved, newest first. `GET /api/cities` lists the city registry; `GET /api/stats` returns the analytics aggregate.

SPA serving. `web/dist/assets` mounts as static files; a catch-all route serves real files if they exist, else `index.html` — which is what makes `/runs/abc123` deep links work on a client-side router. The catch-all is also where **first-party analytics** happens: every non-asset page load logs (path, referrer, user-agent) to SQLite — no third-party script, no cookies.

Part III — The city data pipeline

17. `scripts/build_city.py` — OpenStreetMap -> committed JSON

Design choice: fetch OSM **at build time, not runtime**. The heavyweight geo stack (osmnx/geopandas, ~500 MB of Docker image) is avoided entirely — the script uses raw Overpass queries + stdlib math, and the server just loads JSON.

`CITIES` is the registry: name, country code, bounding box. Adding a city to the project is *one dict entry plus one command*.

Step 1 — roads. One Overpass query fetches every way whose `highway` tag is motorway/trunk/primary/secondary/tertiary (+ links) in the bbox, with all their nodes (`out body; >; out skel qt;`). Major-roads-only keeps a metro area around 8–12 k intersections — dense enough to look real, light enough for an SVG.

Step 2 — projection. Lat/lon -> local kilometers: `x = (lon - lon0) × 111.32·cos(lat_mid)`, `y = (lat0 - lat) × 110.57`, origin at the bbox's top-left with y growing *southward* — the SVG convention, so the frontend never flips anything. Kilometer units also make the A* heuristic admissibility argument trivial (see §14).

Step 3 — simplification. OSM ways are chains of densely-spaced geometry points. The script counts way-membership per node (`usage`); a node kept in the graph is a way *endpoint* or a node used by >= 2 ways (a real intersection). Each way then contributes edges between consecutive *kept* nodes, with weight = the summed segment length ÷ the road class speed, in seconds. Parallel edges keep the faster time. This collapses ~45–85 k raw points to ~8–12 k intersections while preserving true travel distances.

Step 4 — connectivity. A DFS finds the largest connected component and discards the rest (disconnected fragments would make some incidents permanently unreachable). Node IDs are then remapped to dense integers 0...N-1.

Step 5 — stations. Overpass `amenity=fire_station` (nodes, ways and relations via `out center`), each snapped to the nearest kept node, deduplicated by node. If a city has fewer than 10 tagged stations (common outside the US/EU), the script pads by **farthest-point sampling**: repeatedly pick the candidate node that maximizes the minimum distance to every existing station — a greedy k-center that spreads synthetic bases evenly across the map.

Step 6 — landmarks. Named hospitals, universities, rail/metro stations and malls (capped at 150, deduped by name), snapped to nodes. These become the gazetteer for scenario labels ("Paras Hospital", "Netaji Subhash Place").

The output JSON (~0.7–1.1 MB per city) carries city/country/bbox, the projection constants (so the *server* can project incident lat/lons the same way), `nodes` (km coordinates), `latlon` (kept for future geo features), `edges`, `stations`, and `pois`. Overpass etiquette: 5 s sleeps between queries, 10 s between cities, and a User-Agent header.

Part IV — The frontend

React 19 + TypeScript + Vite + Tailwind v4 (via `@tailwindcss/vite` — no PostCSS config) + react-router 7. No chart library, no map library, no state library: the map is hand-rolled SVG and state is `useState`/`useMemo`, which keeps the bundle at ~82 KB gzipped.

18. Foundations

- `vite.config.ts` proxies `/api` (including WebSocket upgrades, `ws: true`) to `localhost:8000` during development; in production FastAPI serves the built files, so there is no CORS story at all.
- `types.ts` mirrors the backend wire format exactly (`Graph`, `Metrics`, `IncidentView`, `UnitView`, `FeedEvent`, `Frame`, `RunParams`, `RunMeta`, `RunDetail`, `CityInfo`). One file, one source of truth for shapes.
- `api.ts` wraps the five endpoints. `createRun` parses FastAPI's `{"detail": ...}` error body so users see "All simulation slots are busy..." rather than "failed to start run (429)". `runSocketUrl` derives `ws://`/`wss://` from `location`, so the same code works in dev, plain HTTP, and TLS.
- `theme.ts` centralizes the two color scales — incident types (Fire orange, Medical rose, Flood sky, Collapse purple, Hazmat lime...) and unit types (Ambulance white, FireTruck red, Police blue...) — plus `fmtSimTime` (mm:ss). Every component imports from here; nothing hard-codes a color twice.
- `index.css` sets the dark theme: slate-950 body with two faint radial gradients (orange top, cyan bottom), thin dark scrollbars, orange selection, and three keyframe animations — `ping-slow` (the incident pulse), `blink` (the LIVE dot), `spin` (the loader).
- `main.tsx` mounts two routes: `/` -> Home, `/runs/:id` -> RunPage.
- `Navbar.tsx` renders the beacon logo mark (`BeaconMark` — three SVG arcs over a dot, matching the favicon) in a sticky, blurred bar, and accepts children so RunPage can slot its share button into the right side.

19. `pages/Home.tsx`

State: the scenario form (`durationMin`, `incidents`, `seed`, `speed`), the city machinery (`cities` from `/api/cities`, `cityKey`, `useReal`), and transient flags. Two derived values do the heavy lifting:

```
const mode = useReal ? cityKey : 'synthetic'
const showScenarioKnobs = !useReal || !selectedCity?.live
```

— the scenario knobs (incident count, seed) appear for the synthetic grid *and* for real cities without a live feed (you design the scenario), but hide for Seattle (reality supplies the incidents).

`start()` POSTs the params and navigates to the run page. `watchDemo()` — the hero CTA — starts a fixed one-minute showcase (30 min / 150 incidents / 60×) and, if the run cap rejects it, silently falls back to the newest completed replay: the demo button *never* dead-ends a first-time visitor.

Layout: hero (badge, two-tone headline, CTA pair) -> the configure card (the real-city selector with LIVE 911 DATA / REAL MAP badges and an explanation of which is which, the synthetic card, then the sliders) -> a three-step pipeline explainer -> recent-runs cards (params summarized per mode, run outcome, and a live/replay badge) -> a footer naming the stack.

20. `pages/RunPage.tsx` — the live/replay state machine

The page holds `frames` (the growing array), `cursor` (which frame is displayed), `status` (`loading | running | complete | not_found`), `playing` (replay), and `following` (live). A ref mirrors `following` so the WebSocket callback always reads the current value without re-subscribing.

The loading effect (keyed on the run id) fetches `GET /api/runs/:id`:

- **complete** -> set the whole timeline, autoplay from frame 0.
- **running** -> open the WebSocket and reduce its messages: `init` sets graph/params; `frames` (catch-up) and `frame` (live) append — and, if following, snap the cursor to the newest frame inside the same state update (no flicker, no double render); `done` flips status to complete (the frames already accumulated *become* the replay in place — the page transitions from live to replay without a reload); `error` means the run finished between the fetch and the connect, so re-fetch the replay.

The cleanup function closes the socket and sets a `cancelled` flag so a stale fetch can't clobber state after navigation.

The playback timer runs only when `status === 'complete' && playing`: a `setInterval` at `1000 / (12 × replaySpeed)` ms advances the cursor and auto-pauses at the end. Scrubbing the slider pauses playback and disengages following; the "Go live" button re-engages it.

Derived data is memoized per cursor move: the event log is the concatenation of `frames[0..cursor].events` (last 30, newest first — so scrubbing backwards "rewinds" the log), and the two sparkline series are `backlog` and `avg_response_min` sliced up to the cursor.

21. `components/CityMap.tsx` — the hand-rolled map

Scaling. `SCALE = min(64, max(34, 950 / max(width, height)))` — the grid city (width 12) hits the 64 cap and looks exactly like the original; Seattle (~25 km tall) lands around 37, filling the same screen area. Everything else is proportional, so one component serves both geometries.

The road layer is memoized. Frames arrive ~12×/second, and a real city has ~9,500 edge `<line>`s — rebuilding that vdom per frame would dwarf everything else. But closures/congestion are the only thing that changes a road's appearance, and they change rarely (never, in real mode). So the component digests them to strings (`closedKey`, `congKey`) — frame arrays are fresh objects every tick, but the *digest* is stable — and `useMemo`s the entire layer on `[graph, closed, congestion]`. Closed roads render red-dashed; congested roads amber with width/opacity scaled by the multiplier; intersection dots are skipped entirely above 600 nodes.

Stations are "S" squares. **Incidents** (active only) render at their node: a `ping-slow` pulse ring while `pending`, a solid circle with radius `6 + severity × 1.8`, a white ring when on-scene, and a label whose `textAnchor` flips to start/end near the map edges so names never clip. A `<title>` gives the full detail tooltip.

Units. Idle units at a station would stack into one dot, so `idleOffsets` fans them out on a golden-angle spiral (`angle = i × 2.39996`, radius stepping outward) — deterministic per frame, no jitter between frames. Moving units draw bigger (r 5 vs 3.5) with full opacity at their interpolated `x, y` from the frame; idle units fade to 45%. Colors come from `theme.ts`, echoed in the legend below the map.

22. The panel components

- `MetricsBar` — nine stat cards from `frame.metrics` (calls, incidents, dupes merged, quarantined, dispatches, backlog, resolved, avg response, lives saved). Backlog turns rose above 5; resolved/lives are green.
- `IncidentFeed` — active incidents sorted by live priority, top 20: color dot, id, location, severity/people, a status pill, and the priority number. The resolved count comes from `metrics.resolved` (resolved incidents aren't in frames — see §13).
- `ResourcePanel` — one stacked bar per fleet: green available, amber dispatched, sky returning, slate out, with an "n/total free" readout.
- `EventLog` — the dispatch narrative, newest first: [!] created (rose), -> dispatched (amber), [y] resolved (green), each with its sim-time stamp.
- `Sparkline` — a 240×44 polyline scaled to its own max; used for backlog and average response. No axes by design: these are trend glances, and the current value is printed beside the label.
- `PlaybackControls` — the transport bar. Live: a pulsing LIVE button that toggles following. Replay: play/pause, the scrub slider (`cursor / total`), and a 0.5–4× speed picker. The sim clock renders as mm:ss.

Part V — Deployment

23. `Dockerfile` — two stages

```
FROM node:25-alpine AS webbuild        # matches the npm that made the lockfile
COPY web/package.json web/package-lock.json ./
RUN npm ci                              # cached unless the lockfile changes
COPY web/ ./ && RUN npm run build

FROM python:3.12-slim
RUN pip install -r requirements.txt
COPY dispatch_grid/ server/ ...
COPY --from=webbuild /app/web/dist web/dist
CMD uvicorn server.app:app --host 0.0.0.0 --port ${PORT}    # shell form: $PORT expands
```

Stage 1 exists only to produce `web/dist`; Node never ships in the runtime image. Copying the lockfile *before* the source maximizes Docker layer caching. The CMD uses shell form deliberately so the host's `$PORT` (Render sets it) expands. `render.yaml` pins `runtime: docker`, the free plan, and `/api/runs` as the health check.

A war story worth keeping: the first deploys failed with a cryptic `npm error code EUSAGE`. Root cause: `package-lock.json` had been generated on macOS, and npm silently omitted Linux-only optional dependencies (`@emnapi/*`, needed by Tailwind's native bindings) — installs worked on the Mac and failed on *every* Linux machine. The fix: regenerate the lockfile inside a Linux container (`npm install --package-lock-only` in `node:25-alpine`). If a future deploy fails the same way after adding a package, do that again.

24. Operational safeguards

- **Run cap** — at most 4 concurrent simulations (`MAX_ACTIVE_RUNS`); excess requests get a polite 429 and the frontend falls back to replays.
- **Feed cache** — the Seattle fetch is cached 3 minutes, so a burst of visitors doesn't hammer data.seattle.gov.
- **Live-feed failure** — a friendly 502 ("try again in a minute, or pick another city"), never a stack trace.
- **Ephemeral disk caveat** — on free-tier hosts `runs.db` (replays *and* analytics) resets on every redeploy. Fine for a demo; a persistent disk or Postgres is the upgrade path if permanent links start to matter.

Design decisions at a glance

Decision	Why
Single-writer ownership per module	conflict resolution by construction — no locks anywhere
Product-form priority with sqrt(lives) and capped aging	no factor dominates; nothing starves
Quarantine + corroboration (0.42 / 420 s)	false reports cost zero dispatches; twice-reported real incidents always admitted
Atomic multi-resource dispatch	partial fills waste scarce units at scenes they can't handle

Decision	Why
12% coverage reserve, 2.2× preemption ratio	fresh incidents always find units; stealing must be clearly justified
Real CAD records enter as `triage.report` events	downstream swarm identical for synthetic, live and scenario data
OSM fetched at build time into committed JSON	no geo stack in the image; no runtime dependency on Overpass
Straight-line cost for candidate *ranking* only	50× faster allocation on 8k-node graphs; final routes stay exact
Frames exclude resolved incidents	late-run frame size stays flat; the count lives in metrics
Memoized SVG road layer keyed on closure digests	9.5k lines render once, not 12×/second
Server-paced WS streaming + SQLite replay	one URL is both the live view and the permanent share link
First-party analytics in SQLite	no third-party script, no cookie banner, one `/api/stats` call

How to extend the project

- **Add a city**: add a `CITIES` entry (name, country, bbox) in `scripts/build_city.py`, run `python3 scripts/build_city.py <key>`, commit the JSON. It appears in the picker automatically.
- **Add a live feed**: write a fetcher in `server/realmode.py` that returns `TriageReport`s, add the city key to `LIVE_FEED_CITIES`, and branch on it in `SimulationSession.__init__` (the Seattle path is the template).
- **Swap the triage brain**: any class honoring `extract(call) -> TriageReport` drops in — `LLMTriageAgent` is the worked example, and `evaluation.py` will score it against the rules.
- **Bigger maps**: raise the Overpass class filter (add `residential`) and swap the SVG for canvas/WebGL rendering; the wire format already carries everything needed.
- **Real fleets**: `REAL_CITY_FLEET` in `server/engine.py` is one dict — fire department annual reports publish real apparatus counts per city.